PDF-версия исходной лекции iSMART из сохранённого HTML. Содержание не заменено пересказом. Служебные элементы интерфейса исключены.

Практический результат

Содержание

- Короткая памятка
- Минимум команд pandas
- Практические задания
- Задание с кодом ИИ
- Парная мини-проверка
- Что сдать
- Итог урока
- Материалы к занятию

Перед подготовкой данных для модели важно убедиться, что таблица открывается правильно, а назначение каждого столбца понятно. На практике это помогает не перепутать входные данные, служебные поля и целевой ответ.

- вы создадите DataFrame из данных, записанных прямо в коде, и проверите его размер;
- откроете CSV-файл через pandas и убедитесь, что данные прочитаны в табличном виде;
- получите количество строк и столбцов таблицы, а также список названий столбцов;
- разделите столбцы на числовые и категориальные с помощью типов данных pandas;
- соберёте список входных признаков для будущей модели, исключив служебные поля и целевой столбец;
- исправите фрагмент кода, сгенерированный ИИ, и объясните причину исправления.

К концу работы нужно уметь не только получить список столбцов программно, но и объяснить, почему одни столбцы подходят для входных признаков, а другие необходимо исключить.

Короткая памятка

DataFrame удобно представлять как обычную таблицу: строки описывают отдельные объекты, а столбцы содержат их свойства. Однако роль столбца определяется не только типом данных. Например, числовой идентификатор записан числами, но обычно не является содержательным признаком объекта.

Элемент	Как понимать
DataFrame	таблица pandas внутри Python; у неё есть строки, столбцы и типы данных
строка	один объект или один пример, например одно обращение или один ученик
столбец	свойство объекта или результат, который требуется определить
числовой столбец	столбец с числами: длина текста, оценка срочности, часы сна; числа могут быть целыми или дробными
категориальный столбец	столбец с категориями: источник, приоритет, группа; значения обозначают варианты, а не измеряемую величину
features / X	столбцы, которые можно подать на вход будущей модели для получения предсказания
target / y	столбец с правильным ответом, который модель должна научиться определять по входным признакам

Один и тот же столбец нельзя одновременно использовать как обычный входной признак и как целевой ответ в рамках одной задачи. Если готовый ответ попадёт в **features**, модель получит информацию, которой не должно быть во входных данных.
Также важно различать категорию и число. Например, значение `request_id = 105` записано числом, но оно лишь обозначает конкретную строку. Арифметические действия с таким номером обычно не описывают свойства обращения, поэтому служебный идентификатор исключают из списка признаков.

Минимум команд pandas

Эти команды позволяют выполнить первичную проверку таблицы до анализа данных. Сначала подключается библиотека и читается файл, затем проверяются первые строки, размер, названия и типы столбцов.

```
import pandas as pd
df = pd.read_csv('support_requests_10_11_spo.csv')
df.head()
df.shape
list(df.columns)
df.select_dtypes(include='number').columns
df.select_dtypes(include='object').columns
```

`import pandas as pd` подключает pandas под коротким именем `pd`. Такое имя затем используется при обращении к функциям библиотеки.
`pd.read_csv(...)` читает CSV-файл и создаёт DataFrame. Имя файла должно совпадать с именем доступного файла, иначе pandas не сможет его найти.
`df.head()` показывает первые строки таблицы. Эта команда помогает проверить, правильно ли распознаны заголовки и распределились ли значения по столбцам. По умолчанию выводятся первые пять строк, но размер всей таблицы эта команда не определяет.
`df.shape` возвращает пару значений в порядке `(количество строк, количество столбцов)`. Например, результат `(12, 10)` означает, что в таблице 12 строк и 10 столбцов. Частая ошибка — поменять эти два числа местами.
`list(df.columns)` преобразует набор названий столбцов в обычный список Python. Порядок элементов совпадает с порядком столбцов в DataFrame.
`select_dtypes(include='number')` выбирает числовые столбцы по типам данных pandas, а `select_dtypes(include='object')` — столбцы типа `object`. Тип `object` часто содержит текстовые значения; категориальность определяется смыслом данных и/или типом `category`. При этом команда проверяет технический тип столбца, но не решает, подходит ли он для модели: это необходимо определить отдельно по смыслу данных.
После `select_dtypes(...)` указано свойство `.columns`, поэтому результатом будут названия выбранных столбцов, а не все их значения.

Практические задания

Задания построены так, чтобы результат можно было проверить автоматически: программа должна вывести конкретную строку. Не добавляйте лишние пояснения в `print`, если работаете в режиме тренажёра. Даже дополнительная подпись перед правильным значением может привести к несовпадению с ожидаемым выводом.
Названия столбцов в заданиях требуется выводить через запятую без пробелов. Для этого список названий можно объединить в строку с помощью `','.join(...)`. Порядок столбцов

необходимо сохранить таким же, как в исходной таблице или в сформированном списке признаков.
В заданиях 4 и 5 классификация выполняется по типам данных в DataFrame. В задании 6 требуется дополнительный смысловой анализ: из автоматически найденных столбцов нужно выбрать только те, которые допустимо использовать как входные признаки.
Например, `request_id` попадёт в список числовых столбцов, потому что хранит числа, но в список признаков задания 6 не войдёт. Столбец `category` относится к категориальным, однако он также исключается, если является ответом, который требуется предсказывать. Исходный столбец `text` тоже не используется в указанном списке: вместо него представлены уже выделенные характеристики текста, например его длина и наличие определённых слов.
Перед отправкой решения полезно проверить три условия: числа в размере таблицы выведены в правильном порядке, названия соединены без лишних пробелов, а список признаков не содержит идентификатор и целевой ответ.

Задание с кодом ИИ

В одном из заданий вы исправляете код, который как будто предложил ИИ. Это не тест на доверие к ИИ, а тренировка внимательности: помощник может предложить синтаксически правильный код, но человек должен проверить, не попал ли в признаки служебный номер или готовый ответ.

Исходный код

```
features = ['student_id', 'age', 'gender', 'country', 'grade', 'sleep_hours',
'screen_time_hours', 'uses_ai_for_study', 'stress_level_1_10',
'exam_preparation', 'wellness_group']
```

В предложенном списке нужно проверить роль каждого элемента. `student_id` является служебным идентификатором ученика, поэтому его следует убрать. `wellness_group` является целевым столбцом: именно его значение должна определять будущая модель, поэтому готовый ответ также нельзя оставлять во входных признаках.
После исправления в списке остаются характеристики ученика, которые используются как входные данные: возраст, пол, страна, оценка, продолжительность сна, экранное время, использование ИИ для учёбы, уровень стресса и подготовка к экзаменам. Их порядок должен соответствовать ожидаемому выводу задания.
Типичная ошибка — удалить только `wellness_group`, но оставить `student_id`, поскольку идентификатор имеет числовой вид. Другая ошибка — удалить все категориальные столбцы.

Категориальный тип сам по себе не делает признак недопустимым: решение об исключении принимается по роли столбца, а не только по его типу.

Парная мини-проверка

После выполнения заданий обменяйтесь с партнёром списками признаков для одной таблицы. Один участник проверяет, нет ли в списке служебного идентификатора, второй — нет ли в списке целевого столбца. Дополнительно проверьте, сохранился ли требуемый порядок названий и нет ли лишних пробелов в строке вывода.
Итог пары: один исправленный список признаков и короткий комментарий, что было изменено. Комментарий должен называть не только удалённый столбец, но и причину удаления, например: «`student_id` исключён, потому что это служебный идентификатор».

Что сдать

- заполненный ноутбук или решения в редакторе платформы;
- выводы программ по каждому заданию в точном формате;
- короткий текстовый вывод: какие столбцы числовые, какие категориальные, какой столбец можно считать целью;
- исправленный список признаков без служебного идентификатора и целевого столбца;
- краткое объяснение, почему числовой тип столбца ещё не означает, что этот столбец подходит для обучения модели.

Итог урока

Таблица становится полезной для ИИ только после того, как человек понял её структуру: где объекты, где столбцы, какие данные числовые, какие категориальные, какие столбцы можно дать на вход будущей модели, а какие нужно убрать. Pandas здесь не обучает модель — он помогает открыть и проверить таблицу, определить типы данных и подготовить список столбцов к дальнейшему анализу.
Автоматический выбор по типу данных является только первым шагом. После него требуется смысловая проверка: идентификатор может быть числовым, цель может быть категориальной, а оба столбца при этом должны быть исключены из **features**. Корректно подготовленный список входных признаков содержит данные, доступные модели для предсказания, но не содержит готового ответа.

Материалы к занятию

- lesson_1_12_table_data_10_11_spo_reworked.pptx (52.7 КБ)

Зачем этот практикум

Содержание

- Материалы к практике

На прошлом занятии вы разобрали базовую схему: объект, признак и ответ. Теперь нужно увидеть эту схему в рабочем виде: в таблице, в Jupyter Notebook и в коде pandas. В этом уроке важен не пересказ понятий, а действие: открыть таблицу, посмотреть структуру, вывести столбцы, отделить входные данные от результата.
Такая последовательность нужна потому, что перед анализом или обучением модели необходимо понять, как устроены исходные данные. Названия столбцов, количество строк и типы значений помогают определить, что именно описывает таблица и какие операции с ней можно выполнять. Если пропустить этот этап, легко выбрать не те признаки или случайно включить столбец с ответом во входные данные.
В табличном наборе данных строки обычно соответствуют отдельным объектам, а столбцы — их признакам. Например, одна строка может описывать одного участника исследования или одно обращение, а значения в её ячейках — характеристики этого объекта. Столбец с результатом выделяют отдельно: именно его требуется анализировать или предсказывать на основе остальных данных.
Во время практикума предстоит выполнить несколько связанных действий:

- загрузить CSV-файл в Jupyter Notebook;
- получить первое представление о размере и структуре таблицы;
- просмотреть названия столбцов и несколько строк данных;
- определить, какие столбцы могут выступать признаками;
- отделить входные данные от столбца с результатом;
- проверить, что после разделения признаки и ответ по-прежнему относятся к одним и тем же объектам.

Важно различать просмотр данных и их изменение. Команда, которая выводит названия столбцов или первые строки, помогает изучить таблицу, но сама по себе не преобразует её. Разделение данных начинается только тогда, когда из исходной таблицы создаются отдельные наборы: набор признаков и набор ответов.
Частая ошибка — считать любой последний столбец таблицы ответом. Положение столбца не определяет его назначение: сначала необходимо понять смысл данных и условие задачи.

Кроме того, столбец с ответом нельзя оставлять среди входных признаков, иначе модель получит значение, которое должна была предсказывать.
После выполнения действий проверьте понимание схемы на выбранной таблице: что является одним объектом, какие столбцы описывают его признаки и какой столбец содержит результат. Ответ должен опираться не только на положение столбцов, но и на смысл их значений.

Материалы к практике

Для работы понадобятся CSV-файлы с табличными данными и стартовый Jupyter Notebook. CSV-файл содержит данные в текстовом табличном формате, а Notebook позволяет последовательно выполнять команды pandas и сразу просматривать результат каждой операции.

- global_high_school_student_lifestyle_wellness_edu_10_11_spo.csv (29.0 КБ) — основной CSV-файл для изучения структуры табличных данных.
- lesson_1_12_table_data_10_11_spo_start.ipynb (6.1 КБ) — стартовый Notebook, в котором выполняются задания практикума.
- support_requests_10_11_spo.csv (1.3 КБ) — дополнительный CSV-файл для повторения тех же действий на другой таблице.

Перед началом работы загрузите CSV-файлы в среду, где будет открыт Notebook. При чтении таблицы важно указать корректный путь к файлу: имя в коде должно совпадать с именем загруженного файла, включая расширение `.csv`.